EVM REVERSE ENGINEERING

Following the Bytecode of Smart Contracts

A practical path from deployed bytes to security-relevant logic

Souhail Mssassi | Head Of Security at Scroll | @mssassi

0x
5B 57 F4

Whitehat bounty signal

USD 10M
Wormhole bridge bug reported by `satya0x`

USD 6M
Aurora critical issue reported by `pwning.eth`

USD 2M
Optimism mint bug reported by Jay Freeman

USD 2M
Polygon bridge issue reported by Gerhard Wagner

Reverse engineer → Build proof → Report privately → Protocol survives

Point The same skills used for exploit development can be used to prevent the exploit from ever reaching mainnet.

Why this matters now

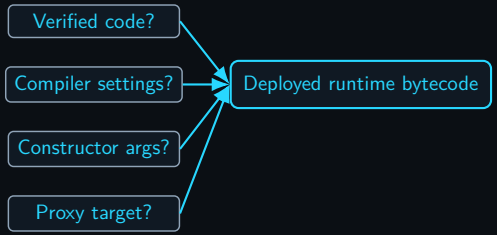
USD 2.87B
stolen across nearly 150 crypto hacks in 2025, according to TRM Labs.

Bridge failures
Ronin, Wormhole, and Nomad showed how cross-chain logic concentrates risk.

USD 15M
maximum listed critical bounty for LayerZero Group 1 V1 smart contract issues.

Bytecode is the ground truth

- Verified code may be unavailable.
- Uploaded code may not match deployed code.
- Proxy contracts may hide the real implementation.
- Optimizer output can look unlike the authored Solidity.
- Constructors and immutables modify runtime behavior.



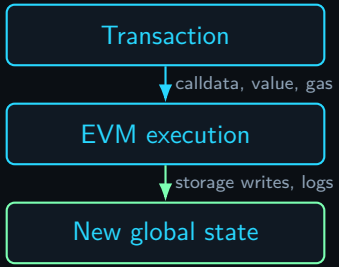
Auditor rule Treat verified Solidity as a helpful witness. Treat deployed bytecode as evidence.

Where reversing pays off

- Unverified contracts
- Incident response after exploitation
- Auditing deployed systems
- Checking code equivalence
- Understanding proxies, clones, and libraries
- Detecting hidden logic
- Malicious tokens and phishing contracts
- Bug bounty triage
- Asset movement path reconstruction
- Privileged function recovery

Blockchain in a nutshell

- A blockchain is a replicated state machine.
- Ethereum state contains accounts, balances, code, and storage.
- A transaction asks the network to execute code and update state.
- Gas meters execution and prevents unbounded computation.

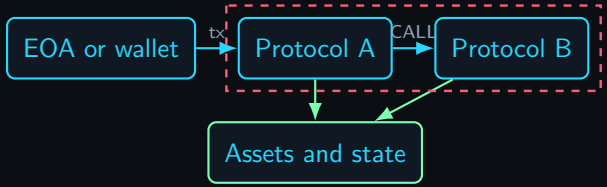


Contract = bytecode
plus persistent storage

Web2 analogy A contract is not a backend you can patch at 2 a.m. It is public code executing under consensus.

Smart contract security model

- Users interact through transactions.
- Contracts manage assets, permissions, and accounting.
- Contracts call other contracts inside the same transaction.
- Audit work is about invariants, trust boundaries, and side effects.



Audit question What must never happen, and which bytecode paths can break it?

Vulnerability classes auditors map to bytecode

State and accounting

- Arithmetic mistakes
- Bad balance updates
- Storage collision
- Signature replay

Authority

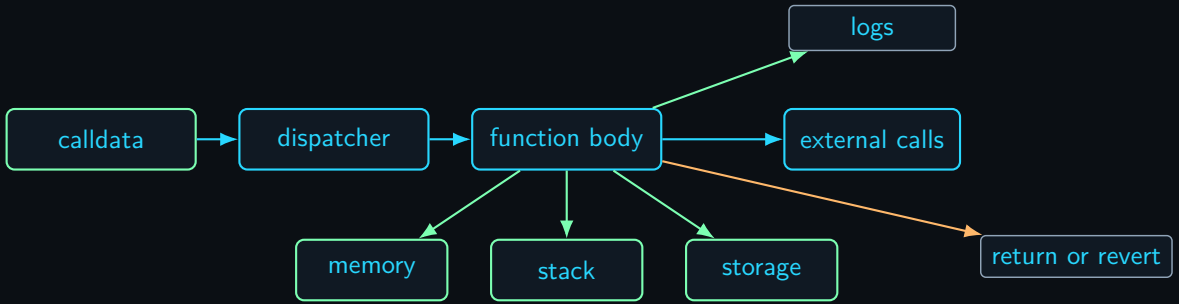
- Access control bugs
- Hidden owner logic
- Proxy upgrade mistakes
- Governance takeover

Interaction

- Reentrancy
- Oracle manipulation
- Unsafe external calls
- Delegatecall hazards

Bytecode lens Every class above leaves traces: `SLOAD`, `SSTORE`, `CALLER`, `CALL`, `DELEGATECALL`, `JUMPI`, constants, and revert paths.

EVM execution components



The EVM is stack-based, uses 256-bit words, and exposes memory, storage, calldata, returndata, logs, and gas-metered opcodes.

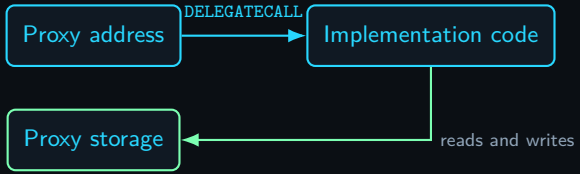
Stack, memory, storage

- `stack`: 1024 items, 256-bit words, last-in first-out.
- `memory`: byte-addressed, temporary, expands with gas cost.
- `storage`: persistent key-value map, 32-byte slots.
- `calldata`: read-only transaction input.

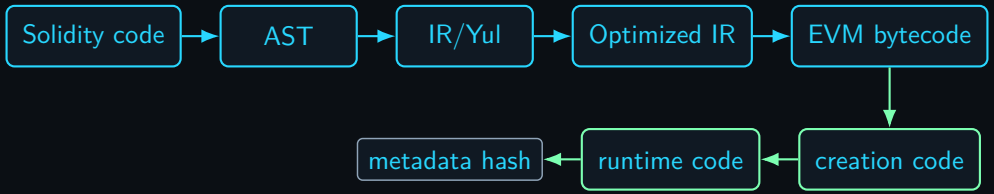
Reverse engineering habit Track the stack around `SLOAD`, `SSTORE`, `CALL`, and `JUMPI`. Those operands explain intent.

Calls and context

Opcode	Code executed	Storage used	Security read
`CALL`	callee code	callee storage	normal external interaction, value may move
`STATICCALL`	callee code	callee storage, read-only	view-style call, no state writes allowed
`DELEGATECALL`	callee code	caller storage	plugin or proxy pattern, high risk if target is attacker-controlled
`CALLCODE`	callee code	caller storage	legacy pattern, almost always suspicious

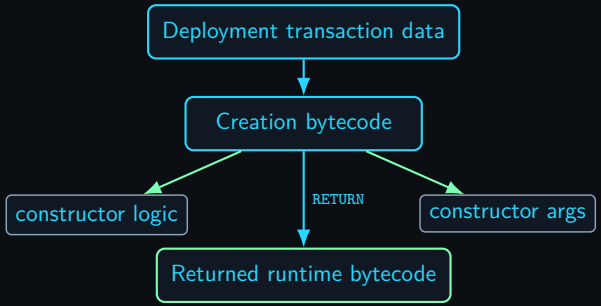


Compilation pipeline



- Same logic can produce different bytecode with different compiler version, optimizer runs, libraries, metadata, or target EVM version.
- Metadata can be a fingerprint, but immutables and libraries need special handling.

Creation code vs runtime code



- Creation code runs once at deployment.
- Runtime code is what remains at the contract address.
- Constructor arguments are appended to creation input.
- Immutable values are embedded into runtime during deployment.
- Metadata is usually appended at the end of runtime bytecode.

Practical check When fetching `eth_getCode`, you get runtime code, not the original creation transaction.

Bytecode anatomy

Runtime bytecode as a byte stream

`PUSH bytes` | `JUMPDEST` | `SLOAD / SSTORE` | selectors, addresses | CBOR tail

- Opcodes are one-byte instructions. `PUSHn` consumes immediate operand bytes.
- Valid dynamic jumps must land on `JUMPDEST`.
- Error selectors, event topics, addresses, constants, and library placeholders become forensic clues.

Annotated bytecode snippet

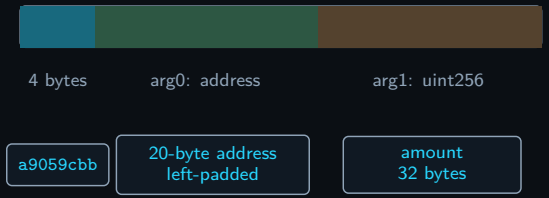
```
0000 PUSH1 0x80
0002 PUSH1 0x40
0004 MSTORE          ; free memory pointer
0005 CALLVALUE
0006 DUP1
0007 ISZERO
0008 PUSH2 0x0010
000b JUMPI           ; reject unexpected ETH
000c PUSH0
000d DUP1
000e REVERT
0010 JUMPDEST
0011 POP
0012 PUSH1 0x04
0014 CALLDATASIZE
0015 LT
0016 PUSH2 0x0042
0019 JUMPI           ; fallback if calldata < 4
001a PUSH0
001b CALLDATALOAD
001c PUSH1 0xe0
001e SHR             ; selector = calldata[0:4]
001f DUP1
0020 PUSH4 0xa9059cbb
0025 EQ
0026 PUSH2 0x0080
0029 JUMPI           ; transfer(address,uint256)
```

- Prologue initializes Solidity memory convention.
- Payability check rejects value unless function permits it.
- Dispatcher extracts the first four bytes.
- `PUSH4` constants are function selector clues.
- `JUMPI` edges begin the control-flow graph.

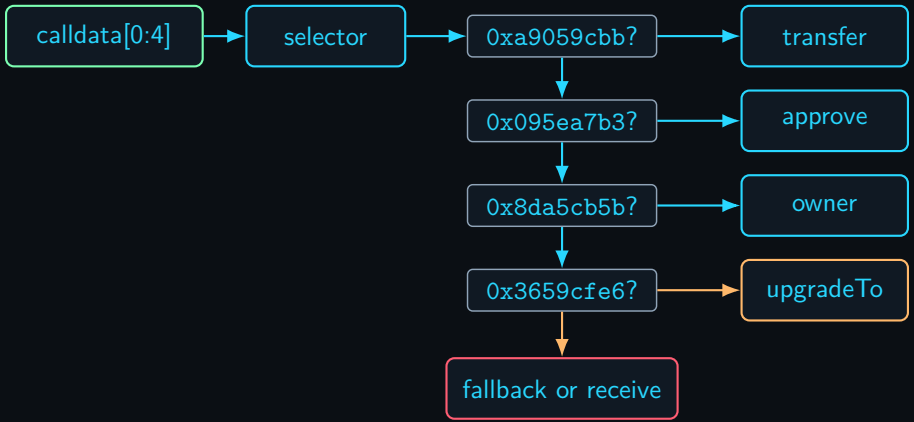
ABI calldata

- External function call data starts with a 4-byte selector.
- Arguments are ABI-encoded in 32-byte words.
- Static values are placed inline.
- Dynamic values use offsets to later data.

`transfer(address,uint256)`
`0xa9059cbb` selector



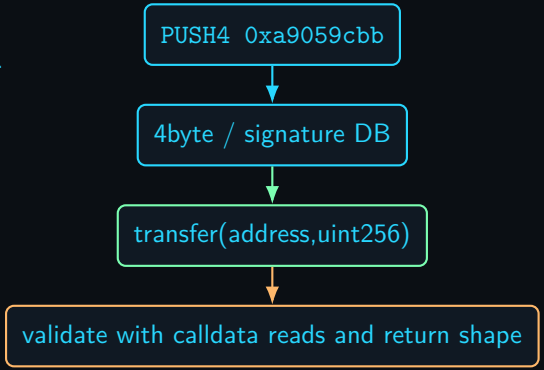
Function dispatcher



Selectors: transfer `a9059cbb`, approve `095ea7b3`, owner `8da5cb5b`, upgradeTo(address) `3659cfe6`.

Selector recovery

1. Find dispatcher region.
2. Extract `PUSH4` constants compared to selector.
3. Query selector databases.
4. Cross-check with ABI decoding shape.
5. Treat names as hypotheses, not facts.



Collision warning A 4-byte selector can map to multiple signatures. The bytecode decides, not the database.

Storage slots and packing

- Storage is a sparse map: `slot key -> 32-byte word`.
- Fixed-size state variables are assigned slots starting at 0.
- Small adjacent values may be packed into one slot.
- Bytecode sees only `SLOAD key` and `SSTORE key, value`.

persistent storage

slot 0: owner address
slot 1: balances mapping base
slot 2: packed flags

Reverse goal Turn anonymous keys into labels: slot 0 = owner, slot 1 = balances mapping base, slot X = implementation pointer.

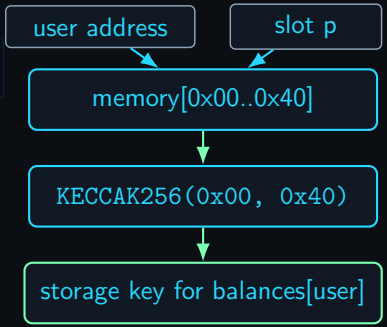
Mapping slots

Solidity pattern

```
mapping(address => uint256) balances; // slot p
balances[user]
```

Storage key

```
keccak256(abi.encode(user, p))
```



Bytecode clue Look for `MSTORE` of a key and base slot, followed by `KECCAK256`, then `SLOAD` or `SSTORE`.

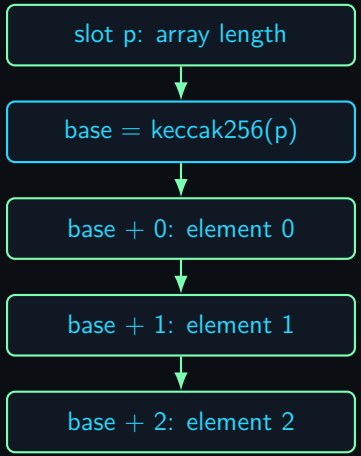
Dynamic arrays and structs

Dynamic array at slot p

- slot p stores length
- data starts at `keccak256(p)`
- element i at `keccak256(p) + i`

Structs

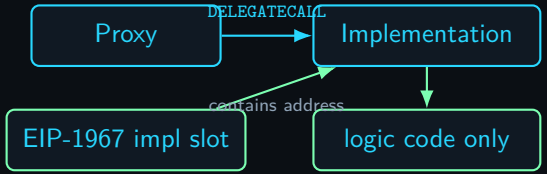
- fields are laid out sequentially
- small fields may pack
- nested dynamic data adds another hash layer



Proxy storage slots

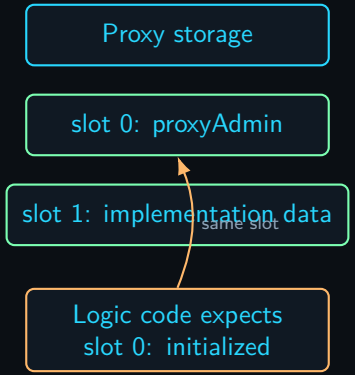
- Proxies keep the address stable and delegate to implementation code.
- EIP-1967 standardizes special slots for implementation, admin, and beacon.
- Implementation slot:

`0x360894a13ba1a3210667c828492db98dca3e2076cc3735a920a3ca505d382bbc`



Audit move Always identify the implementation actually reached by the proxy at the block you care about.

Delegatecall and storage collision risk

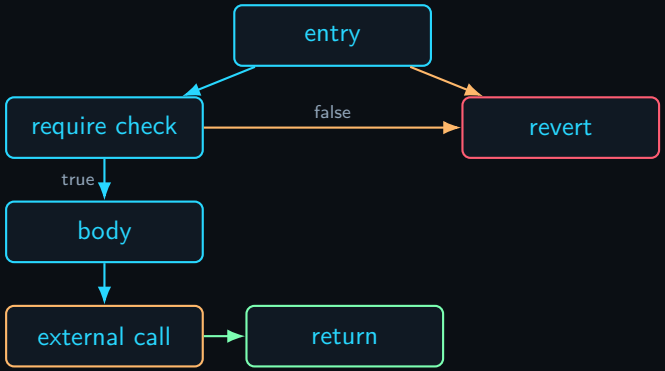


- `DELEGATECALL` executes foreign code in the caller storage context.
- Slot layout compatibility becomes a security property.
- Upgrade code can change meaning without moving funds.
- Storage collisions can reopen initialization or corrupt authority.

Bytecode smell Any path where user-controlled input reaches `DELEGATECALL` target or calldata deserves immediate scrutiny.

Control-flow reconstruction

1. Disassemble bytes into opcodes.
2. Mark valid `JUMPDEST` locations.
3. Split into basic blocks.
4. Add edges from `JUMP` and `JUMPI`.
5. Label entry points, revert paths, loops, and calls.



A Solidity `require` often becomes condition setup, `JUMPI` to success, and a `REVERT` block for failure.

Why decompilers struggle

- Function names are usually gone.
- Variable names are gone.
- Types are inferred, not guaranteed.
- Optimizer can merge, inline, reorder, or delete logic.
- Jump tables and shared revert blocks blur boundaries.
- Proxies split code and storage across addresses.
- Libraries and immutables alter runtime bytes.
- Obfuscation can create misleading control flow.
- Selector collisions and fallback routing confuse names.
- Manual validation is still required.

Use decompilers correctly Let tools propose hypotheses. Use opcode traces, storage formulas, and transaction behavior to prove or reject them.

Manual bytecode workflow

Acquire and normalize

1. Fetch deployed runtime bytecode.
2. Retrieve creation transaction input when possible.
3. Separate creation and runtime code.
4. Strip or decode metadata.
5. Identify compiler and optimizer patterns.
6. Disassemble into opcodes.
7. Locate dispatcher.
8. Recover selectors.
9. Query selector databases.

Recover and audit

10. Reconstruct ABI where possible.
11. Build the control-flow graph.
12. Identify `SLOAD`/`SSTORE` keys.
13. Label storage slots and mappings.
14. Trace external calls and value flow.
15. Identify privileged functions.
16. Find dangerous opcodes.
17. Compare logic with expected behavior.
18. Use decompilers as assistance, not truth.

Toolchain by question

Question	Start with	Validate manually
Deployed code?	RPC, explorer, Sourcify, `cast code`	chain, block, proxy target
Functions?	disassembler, 4byte, decompiler	selector constants, calldata reads
State touched?	traces, storage layout, decompiler	`KECCAK256` patterns, slot labels
Exploit path?	Foundry traces, symbolic tools	constraints, gas, call context
Invariant checks?	Foundry tests, Echidna, Slither	assumptions, fork state, adversarial inputs

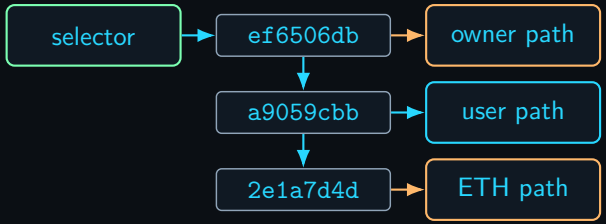
Tool posture A reverse engineer should be able to explain a tool result using opcodes, not just paste the output into a report.

Practical example contract

```
contract VaultToken {
    address public owner;                          // slot 0
    mapping(address => uint256) public balances;   // slot 1

    constructor() { owner = msg.sender; }
    receive() external payable {}
    modifier onlyOwner() {
        require(msg.sender == owner, "not owner");
        _;
    }

    function credit(address user, uint256 amount) external onlyOwner {
        balances[user] += amount;
    }

    function transfer(address to, uint256 amount) external returns (bool) {
        require(balances[msg.sender] >= amount, "balance");
        balances[msg.sender] -= amount;
        balances[to] += amount;
        return true;
    }

    function withdraw(uint256 amount) external {
        require(balances[msg.sender] >= amount, "balance");
        balances[msg.sender] -= amount;
        (bool ok,) = msg.sender.call{value: amount}("");
        require(ok, "eth send");
    }

    function sweep(address to) external onlyOwner {
        (bool ok,) = to.call{value: address(this).balance}("");
        require(ok, "sweep failed");
    }
}
```

Example: dispatcher recovery

Selector	Candidate signature
`8da5cb5b`	`owner()`
`27e235e3`	`balances(address)`
`ef6506db`	`credit(address,uint256)`
`a9059cbb`	`transfer(address,uint256)`
`2e1a7d4d`	`withdraw(uint256)`
`01681a62`	`sweep(address)`



From a bytecode audit perspective, signatures are labels. The proof is the calldata decoding and side effects behind each selector.

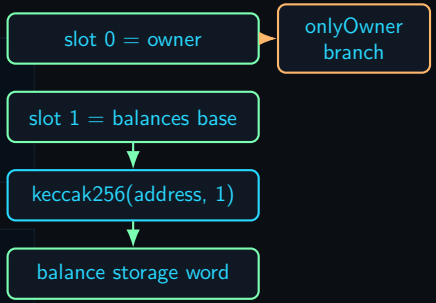
Example: storage recovery

Owner check pattern

```
PUSH0          ; slot 0
SLOAD          ; owner
CALLER         ; msg.sender
EQ
PUSH2 ok
JUMPI
... REVERT ...
ok: JUMPDEST
```

Balances pattern

```
CALLER
PUSH0 MSTORE       ; key
PUSH1 0x01
PUSH1 0x20 MSTORE  ; base slot
PUSH1 0x40 PUSH0 KECCAK256
SLOAD              ; balances[msg.sender]
```



Example: security reasoning from bytes

What the bytes show

- Slot 0 is compared to `CALLER` before `credit` and `sweep`.
- `transfer` touches two mapping keys in slot 1.
- `withdraw` decrements balance before `CALL`.
- `sweep` sends all ETH to an arbitrary address, but only after owner check.

Auditor questions

- Is slot 0 initialized in all deployment paths?
- Is this behind a proxy where slot 0 means something else?
- Can an upgrade bypass the owner check?
- Does any fallback path reach the external call?
- Are token and ETH accounting intentionally separate?

Result The audit finding is not 'the decompiler said so'. It is 'these opcodes move these assets under these conditions'.

Bytecode auditing signals

- `DELEGATECALL`, especially user-controlled target
- `CALL` with nonzero value
- `SELFDESTRUCT` or legacy patterns
- `SSTORE` after external call
- `CALLER` or `ORIGIN` in auth logic
- hardcoded addresses and chain IDs
- `CREATE2` and deterministic deployment paths
- EIP-1967 slots and upgrade functions
- proxy implementation mismatch
- selectors for `upgradeTo`, `initialize`, `sweep`, `execute`
- fallback doing more than routing
- signature verification without nonce or domain separation
- token transfer return values ignored
- strange metadata or unreachable code

Auditor checklist

Authority

- Who can upgrade?
- Who can move assets?
- Is initialization closed?
- Are roles consistent?

State

- What slots are written?
- What mappings hold balances?
- Any storage collisions?
- Are nonces consumed?

Interaction

- What calls leave the contract?
- Is value transferred?
- Can calls reenter?
- Are return values checked?

Report discipline Tie each claim to bytecode evidence: selector, branch, slot, call, and observed trace.

Real-world reversing scenarios

- Unverified token contract with suspicious transfer restrictions
- Verified proxy with unverified implementation
- Malicious airdrop contract requesting approvals
- Exploited DeFi contract during incident response
- Uploaded code that partially verifies but metadata does not match
- Delegatecall-based plugin architecture
- Clone factory with immutable arguments
- Governance execution contract with arbitrary call surface
- Router that hides token movement behind fallback logic
- Contract using custom errors and optimized Yul

Responder mindset Start with what assets can move, then work backward to the branches and storage conditions that permit movement.

Limitations and pitfalls

- Decompiled output is not Solidity code.
- Types, names, and structs are guesses.
- Function boundaries can be ambiguous.
- Optimizer patterns vary by compiler version.
- Dead code may be deliberate bait or compiler artifact.
- Proxies require block-specific context.
- Delegatecall changes storage meaning.
- Libraries and immutables complicate equivalence.
- Obfuscation can waste analyst time.
- Tool false positives and false negatives are normal.

Safety valve When uncertain, reproduce with a local fork or transaction trace. Execution beats speculation.

Learning path: becoming dangerous in the useful way

1. Programming fundamentals
2. Solidity
3. Ethereum and EVM basics
4. Smart contract architecture
5. Common vulnerabilities
6. DeFi primitives
7. Testing with Foundry or Hardhat
8. Reading audits and postmortems
9. Writing local exploits
10. Fuzzing and invariant testing
11. Formal verification basics
12. Bytecode and EVM internals
13. Reversing unverified contracts
14. CTFs and war games
15. Bug bounty report reading
16. Code4rena, Sherlock, Immunefi practice

Reality check The reward is not memorizing exploit names. The reward is building a habit of precise, adversarial reasoning.

Learning links

Core references

- Solidity documentation
- Ethereum Yellow Paper
- evm.codes
- Solidity ABI specification
- EIP-1967 proxy storage slots
- 4byte.directory
- Foundry Book

Security standards and practices

- OWASP SCSVS
- SWC Registry, note stale status
- ConsenSys smart contract best practices

Practice and training

- Ethernaut
- Damn Vulnerable DeFi
- Paradigm CTF infrastructure
- Secureum
- RareSkills
- Code4rena
- Sherlock
- Immunefi

Research and blogs

- Trail of Bits blog
- OpenZeppelin blog

Takeaways and Q&A

- The deployed runtime bytecode is the truth.
- Decompilers are useful, but EVM understanding makes the output auditable.
- Storage and control flow are the heart of reversing.
- Proxies and delegatecall require context, not just code.
- Bytecode reversing is a defender skill: audit, bounty, incident response, and user safety.

Q&A

linkedin.com/in/mssassi
x.com/mss4ssi